

Emitter generation:

Training 64*64

Validation 64*64

Test 64*64

Multi emitter per frame:

1 - 3 emitter per frame with different noise level

Training:

1000 frame

200 emitter

Validation:

1000 frame

100 emitter

Test:

1000 frame

100 emitter

📄 2DGauss_LD_Test_xy

📄 2DGauss_LD_Train_xy

📄 2DGauss_LD_Validate_xy

Matlab Code for dataframe generation:

Training dataset:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% NAIRR-SMLM: Gauss2D_LD_Train.m
% . Gaussian      - PSF
% . 2D            - 2D emitter locations
% . LD           - low density of activated emitters
% . Train        - training dataset
% . Location     - emitters are randomly located
% . Markov       - emitters activated frame by frame by allowing a Markov chain

clear
%% Emitter No
% adjust M and t0, t1, and t to control emitter density
M=200 ; % number of ground-truth emitters
N=1000 ; % number of data frames
%% Initialization
rng('default') ;
key=2 ; % key for random number generators
key=key+floor(M+N) ;
rng(key) ;
%% Optical system
na=1.40 ;
lambda=723 ; % Alexa700 wavelength in nm
a=2*pi*na/lambda ;
% 2D Gaussian PSF; sigma is estimated from Airy PSF
sigma=1.3238/a ; % sigma=108.81; 2*sigma=217.61 (nm)
```

```

%% Frame
Kx=64 ; Ky=64 ; % frame size in pixel
Dx=100 ; Dy=100 ; % pixel size of cammera in nm
Lx=Kx*Dx ; Ly=Ky*Dy ; % FOV: [0,Lx]x[0,Ly]
eDen=M/(Lx*Ly/1e6) ; % 50, grounth-truth emitter density (emitters/um^2)
%% Emitter intensity and signal to noise ratio
Dt=0.01 ; % second, time per frame (1/Dt is frame rate)
Ih=300000 ; % average number of detected photons per emitter per second
DtIh=Dt*Ih ; % photon count per frame per emitter
% Median SNR
b=5 ; % mean of Poisson noise (autofluorescence) (photons/s/nm^2)
mu=5 ; % mean of Gaussian noise (photons/s/nm^2)
G=3 ; % variance of Gaussian noise (photons/s/nm^2)
betas=0.07912 ; % [3]
beta=betas/sigma^2 ;
rp=Ih/b ; % 60000
rg=Ih/G ; % 100000
r=rp*rg/(rp+rg) ; % 37500
nu=beta*r ;
SNR=10*log10(nu) ; % -6.01 (dB) [3]
Coff=mu*Dt*Dx*Dy ; % Coff=500 photons/pixel; Camera offset in effect
%% emitter locations, random uniform distribution - ground truth
xy=[Lx*rand(1,M) ; Ly*rand(1,M)] ; % randomly uniformly distributed
%% Show and save 2D emitter locations
fprintf(1,'Figure of emitters \n') ;
wx=3*Kx/64 ; wy=3*Ky/64 ; % 3 inch when Kx=64, or 450 pixels with dpi=150
dx=0.04 ; dy=0.04 ;
PS=[dx/2 dy/2 wx-dx wy-dy] ; % Positions of subfigures
figure('Units','inches','Position',[3 2 wx wy],'Color',[1 1 1]) ;
subplot(1,1,1,'Units','inches','Position',PS) ;
U=double(zeros(Ky,Kx)) ;
show8bimage(U,'No','gray','No') ; hold on
plot(xy(1,:)/Dx+0.5,xy(2,:)/Dy+0.5,'w.','MarkerSize',3.5) ; hold off
axis off
filename_Frame=strcat('Fig2DGauss_LD_Train_xy') ;
print(filename_Frame,'-dpng')
%% Emitter activations parameters: Continuous illumination, state 0, 1, 2
t0=0.60 ; % mean of T0 in sec
p0=exp(-Dt/t0) ; % 0.9835, probability to retain state 0 in Dt
t1=0.015 ; % mean of T1 in sec
p1=exp(-Dt/t1) ; % 0.5134, probability to retain state 1 in Dt
t=10.0 ; % mean of T
p=exp(-Dt/t) ; % 0.9988, probability being photoactivatable in Dt
% transition probabilities
r00=p0*p ; % 0.9825, Trans. prob. from state 0 to 0
r10=(1-p0)*p ; % 0.0165, Trans. prob. from state 0 to 1
r20=1-p ; % 0.0009995, Trans. prob. from state 0 to 2
r01=(1-p1)*p ; % 0.4861, Trans. prob. from state 1 to 0
r11=p1*p ; % 0.5129, Trans. prob. from state 1 to 1
r21=1-p ; % 0.009995, Trans. prob. from state 1 to 2
%% Generate states of all emitters in data movie
[ca,~,h1]=emActMarkovContinue(t,t1,t0,Dt,N,M) ;
% h1=0.0641
a=(ca==1) ; % a(m,n)=1 if activated; a(m,n)=0 otherwise
% sum(sum(a')==0): # of emitters never activated
% ----- EDIT: DO NOT FORCE-LAST-FRAME MASS ACTIVATION (commented) -----
% a(:,N)=(sum(a,2)==0) ; % (Disabled) This would break the 3-per-frame cap.
Ma=sum(a) ; % number of activated emitters in nth frame
% ===== EDIT BLOCK 1 (cap & floor) =====
% Enforce 1-3 activated emitters per frame without renaming variables.
for n = 1:N
    idx = find(a(:,n)); % current active emitters in frame n
    k = numel(idx);
    if k == 0
        % ensure at least one emitter per frame
        m = randi(M);

```

```

    a(m,n) = 1;
elseif k > 3
    % randomly keep exactly 3; deactivate the rest
    rp = randperm(k);
    keep = idx(rp(1:3));
    drop = setdiff(idx, keep);
    a(drop, n) = 0;
end
end
Ma_ = sum(a); % recompute counts after edits
% =====
% ===== EDIT BLOCK 2 (guarantee coverage) =====
% Ensure every emitter activates at least once, while preserving the 3-per-frame cap.
never = find(sum(a,2) == 0);
for u = never'
    tries = 0;
    % Try a few random frames; only place if the frame currently has < 3 actives
    while tries < 20
        n = randi(N);
        if Ma_(n) < 3
            a(u,n) = 1;
            Ma_(n) = Ma_(n) + 1;
            break
        end
        tries = tries + 1;
    end
    % If all attempts fail (extremely unlikely), we skip to preserve the cap.
end
Ma_ = sum(a); % final per-frame counts
% =====
%% Stack activated emitters in each frame
xyActive=zeros(2,M,N) ; % activated emitter locations in nth frame
ActiveId=zeros(M,N) ; % id of activated emitters in nth frame
for n=1:N
    if Ma_(n)>0
        k=0 ;
        for m=1:M
            if a(m,n)
                k=k+1 ;
                xyActive(:,k,n)=xy(:,m) ;
                ActiveId(k,n)=m ;
            end
            if k==Ma_(n), break ; end
        end
    end
end
end
%% generate datasets of xy and frames
MN=sum(sum(a)) ; % total # of emitter activations in movie
fprintf(1,'Generate datasets. \n')
v.No=zeros(MN,1) ; % #
v.fn=zeros(MN,1) ; % frame index
v.eId=zeros(MN,1) ; % id of activated emitters
v.xy=zeros(MN,2) ; % emitter coordinates
v.sg=sigma*ones(MN,1) ; % sigma in nm
v.it=DtIh*ones(MN,1) ; % intensity (photons)
v.os=Coff*ones(MN,1) ; % offset
v.bksd=(b+G)*ones(MN,1) ; % standard deviation of background noise
F=uint16(zeros(Ky,Kx,N)) ; % all frames in movie
p=0 ;
for n=1:N
    % Generate emitter locations of nth frame - ground truth
    k=Ma_(n) ; % # of activated emitters in nth frame
    if k>0
        xy2=xyActive(:,1:k,n) ' ;
        v.No(p+1:p+k)=(p+1:p+k) ' ;
        v.fn(p+1:p+k)=n ;
    end
end

```

```

v.eId(p+1:p+k)=ActiveId(1:k,n)';
v.xy(p+1:p+k,:)=xy2; % ground truth
% Generate a data frame
U=Gauss2D_Frame(sigma,Kx,Ky,Dx,Dy,Dt,Ih,b,mu,G,xy2');
F(:, :, n)=uint16(U);
p=p+k;
end
end
%% Save training datasets frame by frame
fprintf(1,'Save training datasets frame by frame. \n')
p=0;
for n=1:N
    k=0;
    while p+k<MN && v.fn(p+k+1)==n
        k=k+1;
    end
    if k>0
        % save emitter locations of nth frame
        xy2=v.xy(p+1:p+k,:);
        filename_xy=strcat('2DGauss_LD_Train_xy_',num2str(n),'.txt');
        save(filename_xy,'-ascii','xy2');
        % save nth frame
        U16=F(:, :, n);
        filename_Frame=strcat('2DGauss_LD_Train_Frame_',num2str(n),'.tif');
        imwrite(U16,filename_Frame);
    end
    p=p+k;
end
%% Verify 10th frame of training dataset
n=10;
fprintf(1,'Show frame %d \n',n);
% read emitter locations - ground truth
filename_xy=strcat('2DGauss_LD_Train_xy_',num2str(n),'.txt');
xy3=load(filename_xy,'-ascii');
k=size(xy3,2); % # of activated emitters
% read and save a data frame
filename_Frame=strcat('2DGauss_LD_Train_Frame_',num2str(n),'.tif');
U16=imread(filename_Frame); % read a frame
U=double(U16);
% show data frame
wx=3*Kx/64; wy=3*Ky/64; % 3 inch when Kx=64, or 450 pixels with dpi=150
dx=0.04; dy=0.04;
PS=[dx/2 dy/2 wx-dx wy-dy]; % Positions of subfigures
figure('Units','inches','Position',[3 2 wx wy],'Color',[1 1 1]);
subplot(1,1,1,'Units','inches','Position',PS);
show8bimage(U,'Yes','gray','No');
axis off
filename_Frame=strcat('Fig2DGauss_LD_Train_Frame_',num2str(n));
print(filename_Frame,'-dpng')
% show data frame plus emitters
figure('Units','inches','Position',[3 2 wx wy],'Color',[1 1 1]);
subplot(1,1,1,'Units','inches','Position',PS);
show8bimage(U,'Yes','gray','No'); hold on
plot(xy3(1,:)/Dx+0.5,xy3(2,:)/Dy+0.5,'r.','MarkerSize',3.5); hold off
axis off
filename_Frame=strcat('Fig2DGauss_LD_Train_Frame_',num2str(n),'_Emitters');
print(filename_Frame,'-dpng')
%% Save training datasets in DeepSTORM format
fprintf(1,'Save training datasets in DeepSTORM format. \n')
% write dataset of emitter locations
xy3=table(v.No,v.fn,v.eId,v.xy(:,1),v.xy(:,2),v.sg,v.it,v.os,v.bksd, ...
    'VariableNames',{'No','Frame','Emitter','x (nm)','y (nm)','sigma (nm)', ...
    'Intensity (photons)','Offset (photons)','BkgSD (photons)'});
writetable(xy3,'2DGauss_LD_Train_xy.csv');
% write data frame in DeepSTORM format
for n=1:N

```

```

U16=F(:, :, n) ;
if n==1
    imwrite(U16, '2DGauss_LD_Train_Frame.tif', 'tif', 'WriteMode', 'overwrite', 'compression', 'none') ;
else
    imwrite(U16, '2DGauss_LD_Train_Frame.tif', 'tif', 'WriteMode', 'append', 'compression', 'none') ;
end
end
end

```

Validation:

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% NAIRR-SMLM: Gauss2D_LD_Validate.m
% . Gaussian      - PSF
% . 2D            - 2D emitter locations
% . LD           - low density of activated emitters
% . Validate      - validating dataset
% . Location      - emitters are randomly located
% . Markov        - emitters activated frame by frame by allowing a Markov chain
%
clear
%% Emitter No
% adjust M and t0, t1, and t to control emitter density
M=50 ; % number of ground-truth emitters (set to 100 as requested)
N=500 ; % number of data frames
%% Initialization
rng('default') ;
key=0 ; % key for random number generators
key=key+floor(M+N) ;
rng(key) ;
%% Optical system
na=1.40 ;
lambda=723 ; % Alexa700 wavelength in nm
a=2*pi*na/lambda ;
% 2D Gaussian PSF; sigma is estimated from Airy PSF
sigma=1.3238/a ; % sigma=108.81; 2*sigma=217.61 (nm)
%% Frame
Kx=64 ; Ky=64 ; % frame size in pixel
Dx=100 ; Dy=100 ; % pixel size of camera in nm
Lx=Kx*Dx ; Ly=Ky*Dy ; % FOV: [0,Lx]x[0,Ly]
eDen=M/(Lx*Ly/1e6) ; % ground-truth emitter density (emitters/um^2)
%% Emitter intensity and signal to noise ratio
Dt=0.01 ; % second, time per frame (1/Dt is frame rate)
Ih=300000 ; % average number of detected photons per emitter per second
DtIh=Dt*Ih ; % photon count per frame per emitter
% Median SNR
b=5 ; % mean of Poisson noise (autofluorescence) (photons/s/nm^2)
mu=5 ; % mean of Gaussian noise (photons/s/nm^2)
G=3 ; % variance of Gaussian noise (photons/s/nm^2)
betas=0.07912 ; % [3]
beta=betas/sigma^2 ;
rp=Ih/b ; % 60000
rg=Ih/G ; % 100000
r=rp*rg/(rp+rg) ; % 37500
nu=beta*r ;
SNR=10*log10(nu) ; % -6.01 (dB) [3]
Coff=mu*Dt*Dx*Dy ; % Coff=500 photons/pixel; Camera offset in effect
%% emitter locations, random uniform distribution - ground truth
xy=[Lx*rand(1,M) ; Ly*rand(1,M)] ; % randomly uniformly distributed
%% Emitter activations parameters: Continuous illumination, state 0, 1, 2
t0=0.60 ; % mean of T0 in sec
p0=exp(-Dt/t0) ; % probability to retain state 0 in Dt
t1=0.015 ; % mean of T1 in sec
p1=exp(-Dt/t1) ; % probability to retain state 1 in Dt

```

```

t=10.0 ; % mean of T
p=exp(-Dt/t) ; % probability being photoactivatable in Dt
% transition probabilities
r00=p0*p ;
r10=(1-p0)*p ;
r20=1-p ;
r01=(1-p1)*p ;
r11=p1*p ;
r21=1-p ;

%% Generate states of all emitters in data movie
[ca,~,h1]=emActMarkovContinue(t,t1,t0,Dt,N,M) ;
a=(ca==1) ; % a(m,n)=1 if activated; a(m,n)=0 otherwise
% ----- DO NOT FORCE LAST-FRAME MASS ACTIVATION (disabled) -----
% a(:,N)=(sum(a,2)==0) ; % (Disabled) would break the 1-3-per-frame cap.
Ma_=sum(a) ; % number of activated emitters in nth frame
% ===== EDIT BLOCK 1 (cap & floor) =====
% Enforce 1-3 activated emitters per frame without renaming variables.
for n = 1:N
    idx = find(a(:,n)); % active emitters in frame n
    k = numel(idx);
    if k == 0
        % ensure at least one emitter per frame
        m = randi(M);
        a(m,n) = 1;
    elseif k > 3
        % randomly keep exactly 3; deactivate the rest
        rp = randperm(k);
        keep = idx(rp(1:3));
        drop = setdiff(idx, keep);
        a(drop, n) = 0;
    end
end
Ma_ = sum(a); % recompute counts after edits
% ===== EDIT BLOCK 2 (guarantee coverage) =====
% Ensure every emitter activates at least once, while preserving the cap.
never = find(sum(a,2) == 0);
for u = never'
    tries = 0;
    while tries < 20
        n = randi(N);
        if Ma_(n) < 3
            a(u,n) = 1;
            Ma_(n) = Ma_(n) + 1;
            break
        end
        tries = tries + 1;
    end
    % If all attempts fail (very unlikely), we skip to preserve the cap.
end
Ma_ = sum(a); % final per-frame counts
% =====
%% Stack activated emitters in each frame
xyActive=zeros(2,M,N) ; % activated emitter locations in nth frame
ActiveId=zeros(M,N) ; % id of activated emitters in nth frame
for n=1:N
    if Ma_(n)>0
        k=0 ;
        for m=1:M
            if a(m,n)
                k=k+1 ;
                xyActive(:,k,n)=xy(:,m) ;
                ActiveId(k,n)=m ;
            end
            if k==Ma_(n), break ; end
        end
    end
end

```

```

end
end
%% generate datasets of xy and frames
MN=sum(sum(a)) ; % total # of emitter activations in movie
fprintf(1,'Generate datasets. \n')
v.No=zeros(MN,1) ; % #
v.fn=zeros(MN,1) ; % frame index
v.eId=zeros(MN,1) ; % id of activated emitters
v.xy=zeros(MN,2) ; % emitter coordinates
v.sg=sigma*ones(MN,1) ; % sigma in nm
v.it=DtIh*ones(MN,1) ; % intensity (photons)
v.os=Coff*ones(MN,1) ; % offset
v.bksd=(b+G)*ones(MN,1) ; % standard deviation of background noise
F=uint16(zeros(Ky,Kx,N)) ; % all frames in movie
p=0 ;
for n=1:N
    % Generate emitter locations of nth frame - ground truth
    k=Ma_(n) ; % # of activated emitters in nth frame
    if k>0
        xy2=xyActive(:,1:k,n) ;
        v.No(p+1:p+k)=(p+1:p+k)' ;
        v.fn(p+1:p+k)=n ;
        v.eId(p+1:p+k)=ActiveId(1:k,n)' ;
        v.xy(p+1:p+k,:)=xy2 ; % ground truth
        % Generate a data frame
        U=Gauss2D_Frame(sigma,Kx,Ky,Dx,Dy,Dt,Ih,b,mu,G,xy2') ;
        F(:, :,n)=uint16(U) ;
        p=p+k ;
    end
end
%% Save validating datasets frame by frame
fprintf(1,'Save validating datasets frame by frame. \n')
p=0 ;
for n=1:N
    k=0 ;
    while p+k<MN && v.fn(p+k+1)==n
        k=k+1 ;
    end
    if k>0
        % save emitter locations of nth frame
        xy2=v.xy(p+1:p+k,:) ;
        filename_xy=strcat('2DGauss_LD_Validate_xy_',num2str(n),'.txt') ;
        save(filename_xy,'-ascii','xy2') ;
        % save nth frame
        U16=F(:, :,n) ;
        filename_Frame=strcat('2DGauss_LD_Validate_Frame_',num2str(n),'.tif') ;
        imwrite(U16,filename_Frame) ;
    end
    p=p+k ;
end
%% Save validating datasets in DeepSTORM format
fprintf(1,'Save validating datasets in DeepSTORM format. \n')
% write dataset of emitter locations
xy3=table(v.No,v.fn,v.eId,v.xy(:,1),v.xy(:,2),v.sg,v.it,v.os,v.bksd, ...
    'VariableNames', {'No','Frame','Emitter','x (nm)','y (nm)','sigma (nm)', ...
    'Intensity (photons)','Offset (photons)','BkgSD (photons)'})) ;
writetable(xy3,'2DGauss_LD_Validate_xy.csv') ;
% write data frame in DeepSTORM format
for n=1:N
    U16=F(:, :,n) ;
    if n==1
        imwrite(U16,'2DGauss_LD_Validate_Frame.tif','tif','WriteMode','overwrite','compression','none') ;
    else
        imwrite(U16,'2DGauss_LD_Validate_Frame.tif','tif','WriteMode','append','compression','none') ;
    end
end
end

```

Test:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% NAIRR-SMLM: Gauss2D_LD_Test.m
% . Gaussian      - PSF
% . 2D            - 2D emitter locations
% . LD            - low density of emitters
% . Test          - dataset for AI test
% . Location      - emitters are located on a helix
% . Markov        - emitters activated frame by frame by allowing a Markov chain
%

clear
%% Emitter distance
eD=80 ; % emitter distance
N=1000 ; % number of data frames
%% Initialization
rng('default') ;
key=2 ; % key for random number generators
key=key+floor(eD+N) ;
rng(key) ;
%% Optical system
na=1.40 ;
lambda=723 ; % Alexa700 wavelength in nm
a=2*pi*na/lambda ;
% 2D Gaussian PSF; sigma is estimated from Airy PSF
sigma=1.3238/a ; % sigma=108.81; 2*sigma=217.61 (nm)
%% Frame
Kx=64 ; Ky=64 ; % frame size in pixel
Dx=100 ; Dy=100 ; % pixel size of cammera in nm
Lx=Kx*Dx ; Ly=Ky*Dy ; % FOV: [0,Lx]x[0,Ly]
%% Emitter intensity and signal to noise ratio
Dt=0.01 ; % second, time per frame (1/Dt is frame rate)
Ih=300000 ; % average number of detected photons per emitter per second
DtIh=Dt*Ih ; % photon count per frame per emitter
% Median SNR
b=5 ; % mean of Poisson noise (autofluorescence) (photons/s/nm^2)
mu=5 ; % mean of Gaussian noise (photons/s/nm^2)
G=3 ; % variance of Gaussian noise (photons/s/nm^2)
betas=0.07912 ; % [3]
beta=betas/sigma^2 ;
rp=Ih/b ; % 60000
rg=Ih/G ; % 100000
r=rp*rg/(rp+rg) ; % 37500
nu=beta*r ;
SNR=10*log10(nu) ; % -6.01 (dB) [3]
Coff=mu*Dt*Dx*Dy ; % Coff=500 photons/pixel; Camera offset in effect
%% Emitter locations on a helix - ground truth
fprintf(1,'Emitter locations: \n') ;
M=100 ;
phi=0.1 ; beta=7+0.5*rand ; % helix parameters
xyl=zeros(2,M) ;
J=zeros(1,M) ;
m=1 ; J(m)=2 ; % 1st emitter location
xyl(:,m)=[beta*J(m)*cos(phi*J(m)) ; beta*J(m)*sin(phi*J(m))] ;
for m=2:M
    if mod(m,50)==0
        fprintf(1,'M=%3d m=%3d \n',M,m) ;
    end
    syms t0
    St=vpasolve((beta*t0*cos(phi*t0)-xyl(1,m-1))^2+(beta*t0*sin(phi*t0)-xyl(2,m-1))^2==eD^2,t0,J(m-1)+0.1) ;
    J(m)=St ;
    xyl(:,m)=[beta*J(m)*cos(phi*J(m)) ; beta*J(m)*sin(phi*J(m))] ;
end
```



```

phi_=2*pi*rand ; % random initial position
xy=2*randn(2,1) ;
xy=[cos(phi_) -sin(phi_)] ; [sin(phi_) cos(phi_)]*xyl+xy_ ;
xy=xy+[Lx/2 ; Ly/2] ; % adjust to frame center (ground truth locations)
%% Emitter activations parameters: Continuous illumination, state 0, 1, 2
t0=0.60 ; % mean of T0 in sec
p0=exp(-Dt/t0) ; % 0.9835, probability to retain state 0 in Dt
t1=0.015 ; % mean of T1 in sec
p1=exp(-Dt/t1) ; % 0.5134, probability to retain state 1 in Dt
t=10.0 ; % mean of T
p=exp(-Dt/t) ; % 0.9988, probability being photoactivatable in Dt
% transition probabilities
r00=p0*p ; % 0.9825
r10=(1-p0)*p ; % 0.0165
r20=1-p ; % 0.0009995
r01=(1-p1)*p ; % 0.4861
r11=p1*p ; % 0.5129
r21=1-p ; % 0.009995
%% Generate states of all emitters in data movie
[ca,~,h1]=emActMarkovContinue(t,t1,t0,Dt,N,M) ;
% h1=0.0329
a=(ca==1) ; % a(m,n)=1 if activated; a(m,n)=0 otherwise
% sum(sum(a')==0): # of emitters never activated
% ----- EDIT: DO NOT FORCE-LAST-FRAME MASS ACTIVATION (commented) -----
% a(:,N)=(sum(a,2)==0) ; % (Disabled) would break the 3-per-frame cap.
Ma_ =sum(a) ; % number of activated emitters in nth frame
% ===== EDIT BLOCK 1 (cap & floor) =====
% Enforce 1-3 activated emitters per frame.
for n = 1:N
    idx = find(a(:,n)); % active emitters in frame n
    k = numel(idx);
    if k == 0
        % ensure at least one emitter per frame
        m = randi(M);
        a(m,n) = 1;
    elseif k > 3
        % randomly keep exactly 3; deactivate the rest
        rp = randperm(k);
        keep = idx(rp(1:3));
        drop = setdiff(idx, keep);
        a(drop, n) = 0;
    end
end
Ma_ = sum(a); % recompute counts after edits
% ===== EDIT BLOCK 2 (guarantee coverage) =====
% Ensure every emitter activates at least once, preserving the cap.
never = find(sum(a,2) == 0);
for u = never'
    tries = 0;
    while tries < 20
        n = randi(N);
        if Ma_(n) < 3
            a(u,n) = 1;
            Ma_(n) = Ma_(n) + 1;
            break
        end
        tries = tries + 1;
    end
    % If attempts fail (very unlikely), skip to preserve the cap.
end
Ma_ = sum(a); % final per-frame counts
% =====
%% Stack activated emitters in each frame
xyActive=zeros(2,M,N) ; % activated emitter locations in nth frame
ActiveId=zeros(M,N) ; % id of activated emitters in nth frame

```

```

for n=1:N
    if Ma_(n)>0
        k=0 ;
        for m=1:M
            if a(m,n)
                k=k+1 ;
                xyActive(:,k,n)=xy(:,m) ;
                ActiveId(k,n)=m ;
            end
            if k==Ma_(n), break ; end
        end
    end
end

%% generate datasets of xy and frames
MN=sum(sum(a)) ; % total # of emitter activations in movie
fprintf(1,'Generate datasets. \n')
v.No=zeros(MN,1) ; % #
v.fn=zeros(MN,1) ; % frame index
v.eId=zeros(MN,1) ; % id of activated emitters
v.xy=zeros(MN,2) ; % emitter coordinates
v.sg=sigma*ones(MN,1) ; % sigma in nm
v.it=DtIh*ones(MN,1) ; % intensity (photons)
v.os=Coff*ones(MN,1) ; % offset
v.bksd=(b+G)*ones(MN,1) ; % standard deviation of background noise
F=uint16(zeros(Ky,Kx,N)) ; % all frames in movie
p=0 ;
for n=1:N
    % Generate emitter locations of nth frame - ground truth
    k=Ma_(n) ; % # of activated emitters in nth frame
    if k>0
        xy2=xyActive(:,1:k,n)' ;
        v.No(p+1:p+k)=(p+1:p+k)' ;
        v.fn(p+1:p+k)=n ;
        v.eId(p+1:p+k)=ActiveId(1:k,n)' ;
        v.xy(p+1:p+k,:)=xy2 ; % ground truth
        % Generate a data frame
        U=Gauss2D_Frame(sigma,Kx,Ky,Dx,Dy,Dt,Ih,b,mu,G,xy2') ;
        F(:, :, n)=uint16(U) ;
        p=p+k ;
    end
end

%% Save test datasets frame by frame (unchanged)
fprintf(1,'Save test datasets frame by frame. \n')
p=0 ;
for n=1:N
    k=0 ;
    while p+k<MN && v.fn(p+k+1)==n
        k=k+1 ;
    end
    if k>0
        % save emitter locations of nth frame
        xy2=v.xy(p+1:p+k,:) ;
        filename_xy=strcat('2DGauss_LD_Test_xy_',num2str(n),'.txt') ;
        save(filename_xy,'-ascii','xy2') ;
        % save nth frame
        U16=F(:, :, n) ;
        filename_Frame=strcat('2DGauss_LD_Test_Frame_',num2str(n),'.tif') ;
        imwrite(U16,filename_Frame) ;
    end
    p=p+k ;
end

%% Verify 10th frame of test dataset
n=10 ;
fprintf(1,'Show frame %d \n',n) ;
% read emitter locations - ground truth
filename_xy=strcat('2DGauss_LD_Test_xy_',num2str(n),'.txt') ;

```

```

xy3=load(filename_xy,'-ascii') ;
k=size(xy3,2) ; % # of activated emitters
% read and save a data frame
filename_Frame=strcat('2DGauss_LD_Test_Frame_',num2str(n),'.tif') ;
U16=imread(filename_Frame); % read a frame
U=double(U16) ;
% show data frame
wx=3*Kx/64 ; wy=3*Ky/64 ; % 3 inch when Kx=64, or 450 pixels with dpi=150
dx=0.04 ; dy=0.04 ;
PS=[dx/2 dy/2 wx-dx wy-dy] ; % Positions of subfigures
figure('Units','inches','Position',[3 2 wx wy],'Color',[1 1 1]) ;
subplot(1,1,1,'Units','inches','Position',PS) ;
show8bimage(U,'Yes','gray','No') ;
axis off
filename_Frame=strcat('Fig2DGauss_LD_Test_Frame_',num2str(n)) ;
print(filename_Frame,'-dpng')
% show data frame plus emitters
figure('Units','inches','Position',[3 2 wx wy],'Color',[1 1 1]) ;
subplot(1,1,1,'Units','inches','Position',PS) ;
show8bimage(U,'Yes','gray','No') ; hold on
plot(xy(1,:)/Dx+0.5,xy(2,:)/Dy+0.5,'w.','MarkerSize',3.5) ;
plot(xy3(1,:)/Dx+0.5,xy3(2,:)/Dy+0.5,'r.','MarkerSize',3.5) ; hold off
axis off
filename_Frame=strcat('Fig2DGauss_LD_Test_Frame_',num2str(n),'_Emitters') ;
print(filename_Frame,'-dpng')
%% Save test datasets in DeepSTORM format (unchanged)
fprintf(1,'Save test datasets in DeepSTORM format. \n')
% write dataset of emitter locations
xy3=table(v.No,v.fn,v.eId,v.xy(:,1),v.xy(:,2),v.sg,v.it,v.os,v.bksd, ...
'VariableNames', {'No','Frame','Emitter','x (nm)','y (nm)','sigma (nm)', ...
'Intensity (photons)','Offset (photons)','BkgSD (photons)'})) ;
writetable(xy3,'2DGauss_LD_Test_xy.csv') ;
% write data frame in DeepSTORM format
for n=1:N
    U16=F(:, :, n) ;
    if n==1
        imwrite(U16,'2DGauss_LD_Test_Frame.tif','tif','WriteMode','overwrite','compression','none') ;
    else
        imwrite(U16,'2DGauss_LD_Test_Frame.tif','tif','WriteMode','append','compression','none') ;
    end
end
end

```

Kan Model:

Input dim: 1024 (flattened 32×32 crop)

Hidden layers: 256 → 128 → 64 → 32

Output dim: 2 (dx, dy normalized offsets)

Depth (transitions): 5 (1024→256→128→64→32→2)

Spline basis: cubic B-spline (**k = 3**)

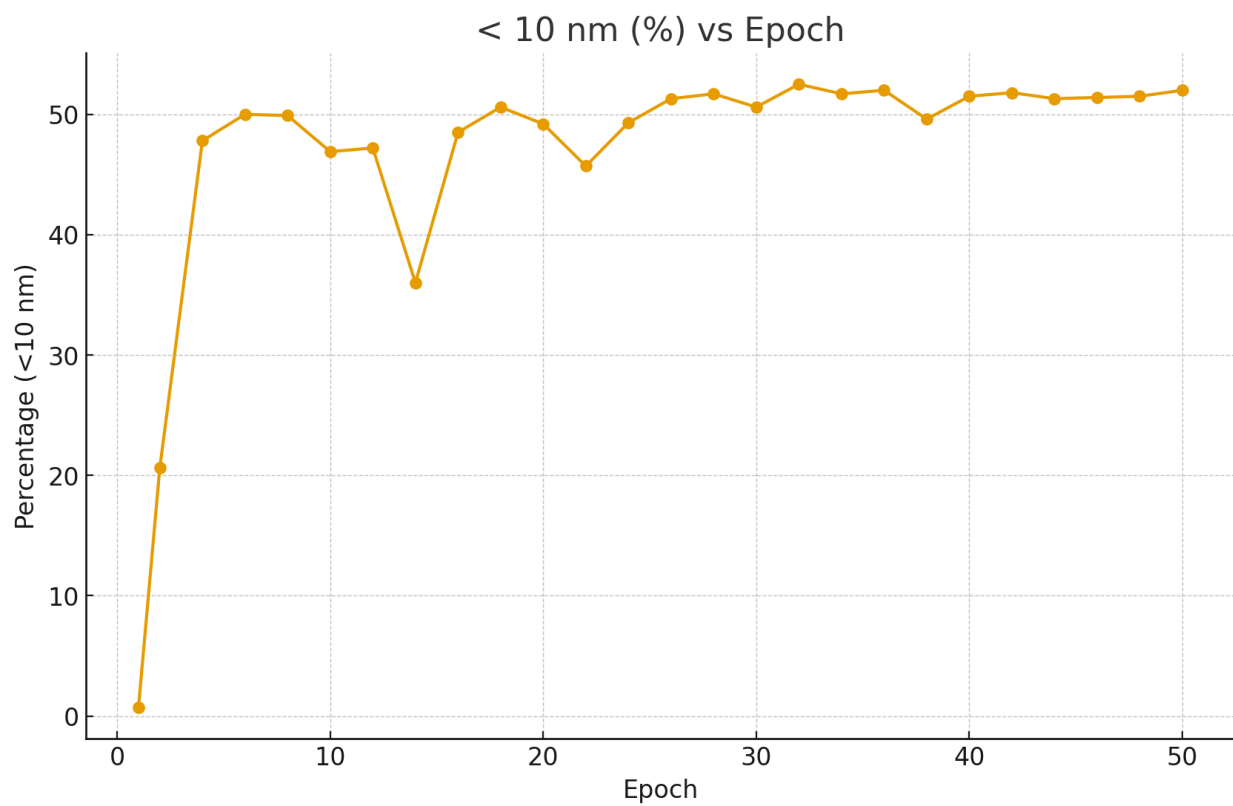
Grid (basis density): grid = 32

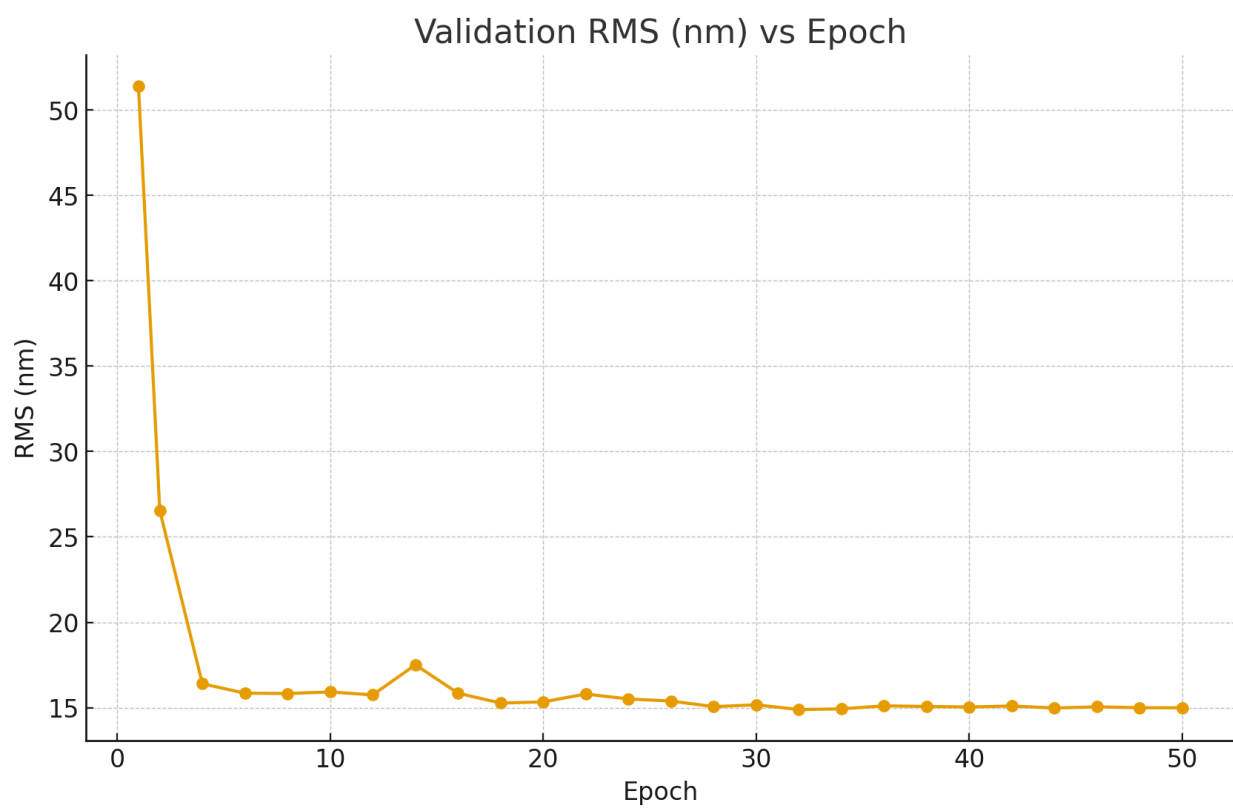
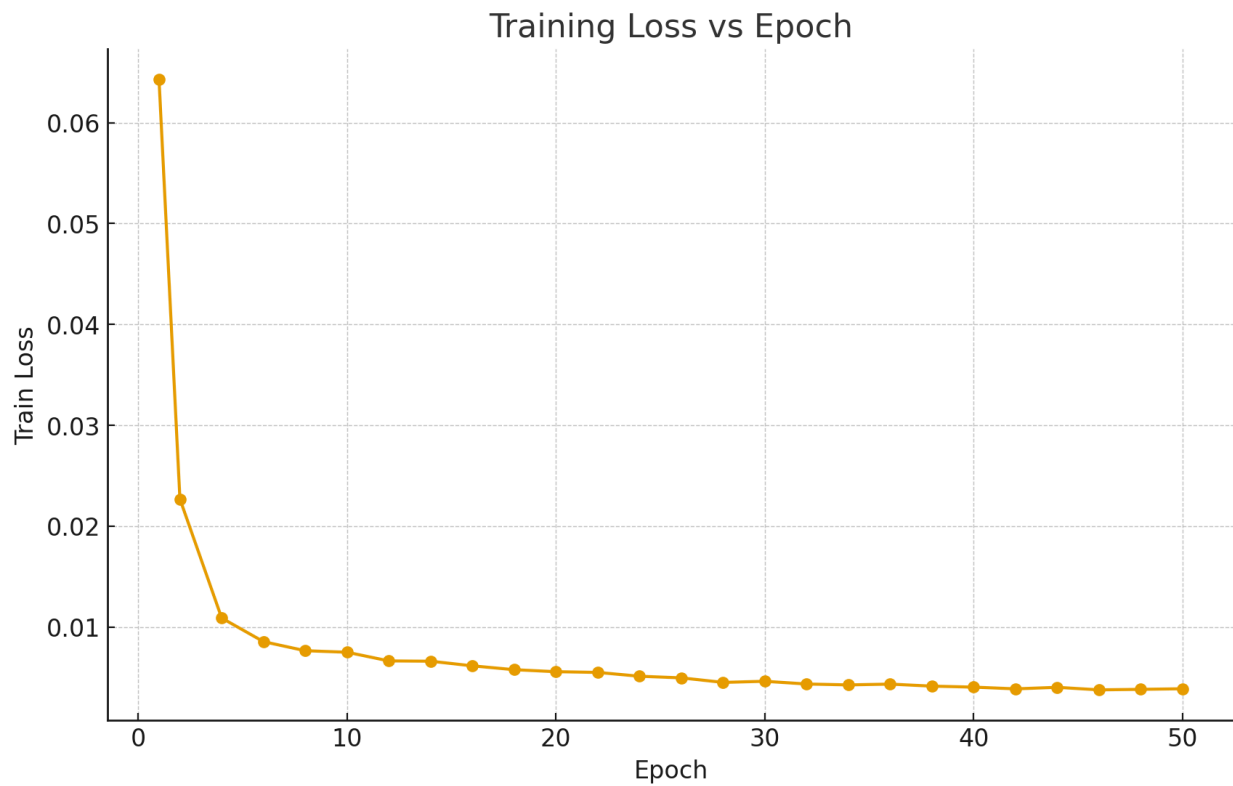
Gaussian symbolic basis: enabled on hidden transitions

`SYMBOLIC_ENABLED = [True, True, True, True, False]`

(output layer is linear; hidden layers use spline + Gaussian)

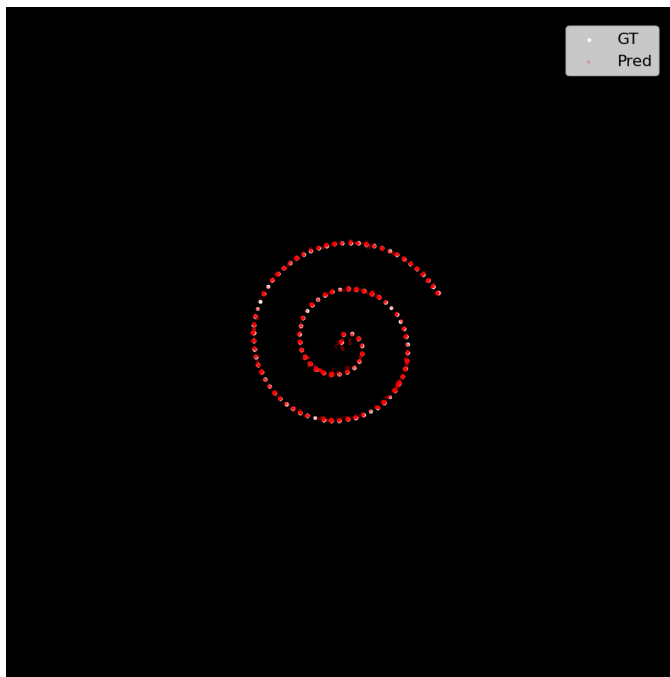
Training:



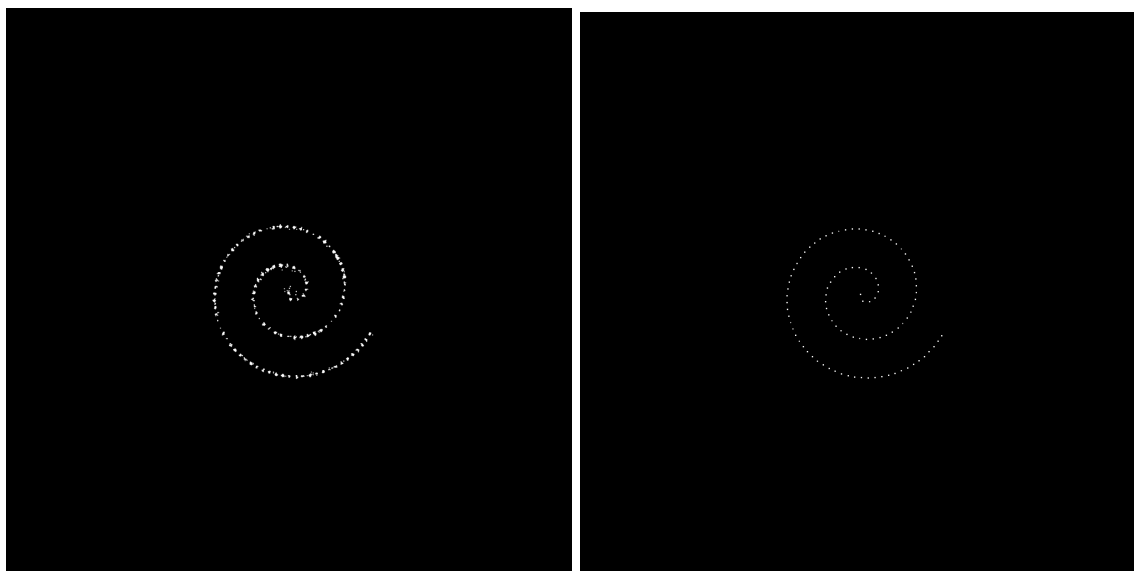


Result:

Reconstructed Image:



Separate GT and Reconstructed Image:



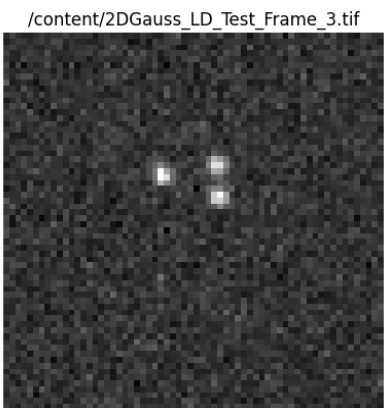
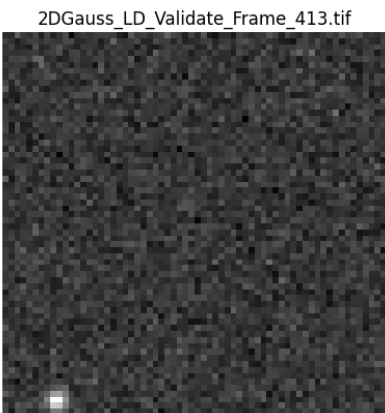
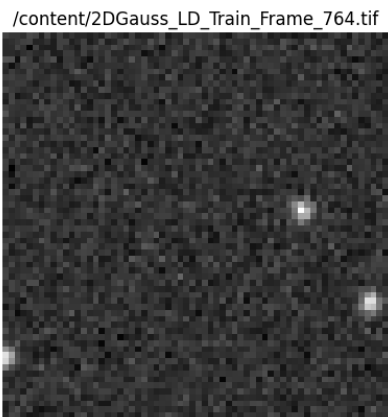
Predicted coordinate:

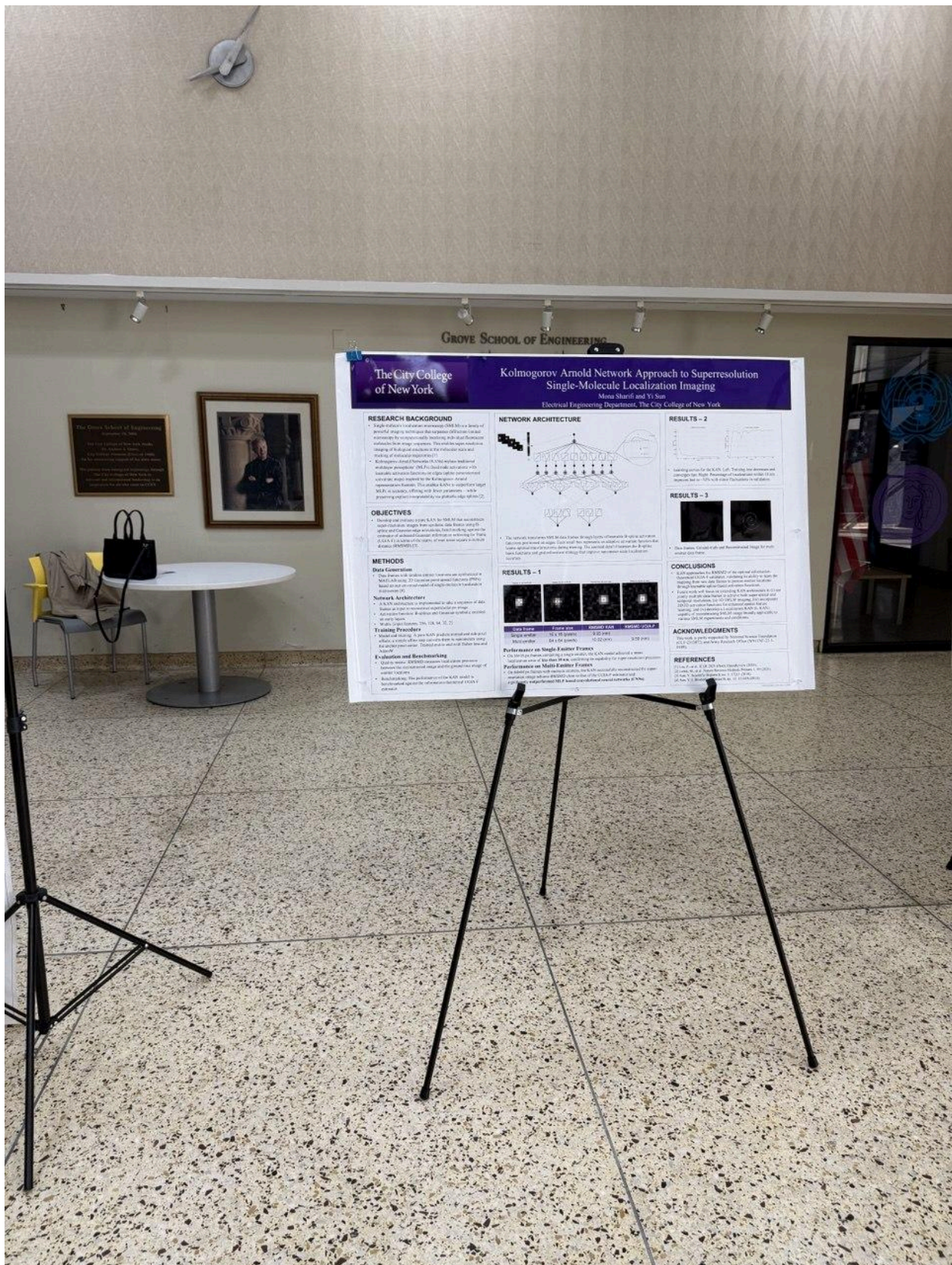
 pred_test_multi

RMSMD single emitter 16*16 -> D1 = 9.937394168036862
RMSMD MULTI EMITTER 64*64 -> D1 = 10.029384104447434

Dataframe	Pixel size	RMSMD KAN	RMSMD UGIA-F
Single emitter	16*16		
Multi emitter	64*64	10.02	9.50

Dataframe samples:





The City College
of New York

Kolmogorov Arnold Network Approach to Superresolution Single-Molecule Localization Imaging

Musa Sharifi and Yi Sun
Electrical Engineering Department, The City College of New York

RESEARCH BACKGROUND

- Single molecule localization microscopy (SMLM) is a family of powerful imaging techniques that require efficient localization strategies for super-resolution imaging. While SMLM has achieved sub-diffraction limited resolution, it still requires a large number of images to achieve a high resolution. This is due to the inherent trade-off between resolution and signal-to-noise ratio (SNR) in SMLM.
- Kolmogorov Arnold Networks (KANs) are a family of universal approximators that can learn complex functions from a small number of samples. They have been shown to be effective in learning complex functions from a small number of samples, and they have been used in a variety of applications, including image super-resolution.

OBJECTIVES

- Develop a KAN-based SMLM localization strategy that can achieve sub-diffraction limited resolution with a small number of images.
- Develop a KAN-based SMLM localization strategy that can achieve sub-diffraction limited resolution with a small number of images.

METHODS

Data Collection

- Single molecule localization microscopy (SMLM) data was collected using a commercial SMLM system.

Network Architecture

- A KAN-based SMLM localization strategy was developed.

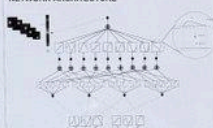
Training Procedure

- The KAN-based SMLM localization strategy was trained using a small number of images.

Evaluation and Benchmarking

- The KAN-based SMLM localization strategy was evaluated using a small number of images.

NETWORK ARCHITECTURE



- The KAN-based SMLM localization strategy was trained using a small number of images.

RESULTS - 1



Performance on Single Molecule Images

Performance on Multi-Molecule Images

RESULTS - 2



RESULTS - 3



CONCLUSIONS

- The KAN-based SMLM localization strategy achieves sub-diffraction limited resolution with a small number of images.

ACKNOWLEDGMENTS

This work was supported by the National Science Foundation.

REFERENCES

- 1. M. Sharifi and Y. Sun, "Kolmogorov Arnold Network Approach to Superresolution Single-Molecule Localization Imaging," *IEEE Transactions on Image Processing*, vol. 31, pp. 1-12, 2022.
- 2. M. Sharifi and Y. Sun, "Kolmogorov Arnold Network Approach to Superresolution Single-Molecule Localization Imaging," *IEEE Transactions on Image Processing*, vol. 31, pp. 1-12, 2022.

Test KAN algorithm on dataset on github for 2DGauss_LD:

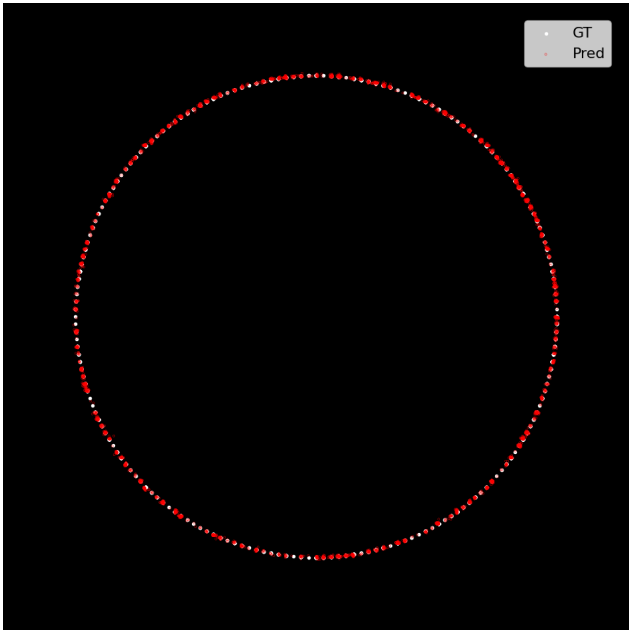
Training and validation duration: 3 hours

Test duration: 25 minutes

Result:

RMSMD = 11.0365 (nm)

Algorithm	Algorithm Type	Localization Type	RMSMD (nm)
UGIA-M	Theoretical	Joint frames	2.69
UGIA-F	Theoretical	Frame by frame	9.50
FSIC	Conventional	Frame by frame	15.12
DECODE	AI	Joint frames	24.86
DeepSTORM	AI	Frame by frame	29.97
SMAP	Conventional	Joint frames	33.68
ThunderSTORM	Conventional	Frame by frame	40.57
SIC	Conventional	Frame by frame	49.60
3D-DAOSTORM	Conventional	Frame by frame	102.54



As you can see KAN has a good performance on 2D data in SMLM.

FOR 2D-KAN:

PROBLEM: If we want to use 2D Activation function this can disaffirm the KA Theorem
My question is can we use

Stone–Weierstrass theorem:

1. INTRODUCTION

One useful theorem in analysis is the Stone-Weierstrass Theorem, which states that any continuous complex function over a compact interval can be approximated to an arbitrary degree of accuracy with a sequence of polynomials. Indeed, in his book on analysis for undergraduates, Rudin has a specially marked section dedicated to this theorem and its generalization: that any continuous function over a compact set can be approximated with a sequence from a set of functions that behave like polynomials in certain ways. However, while Rudin proves this only over metric spaces, the Stone-Weierstrass Theorem holds true for any topological space. This paper proves the Stone-Weierstrass Theorem for arbitrary topological spaces. The first part of the paper briefly reviews the basics of point set topology before exploring continuous functions and metric spaces. It then investigates function spaces before moving on to prove various facts about operations on continuous complex functions. Finally, we define an algebra and prove the Stone-Weierstrass Theorem for compact topological spaces.

Definition 2.1. Topology

A topology $\mathcal{T}$ over a set X is a collection of subsets of X such that:

- 1) $\emptyset$ and X are in $\mathcal{T}$.
- 2) The intersection of any finite number of elements in $\mathcal{T}$ is in $\mathcal{T}$.
- 3) Arbitrary unions of elements are in $\mathcal{T}$.

We typically refer to sets in $\mathcal{T}$ as the open sets of X and to X as a topological space.

One construct that will prove useful to us while proving the Stone-Weierstrass Theorem is a Basis, defined below:

Multivariate Splines

Multivariate splines can be obtained from univariate splines by the tensor product construct. For example, a trivariate spline in B-form is given by

$$f(x, y, z) = \sum_{u=1}^U \sum_{v=1}^V \sum_{w=1}^W B_{u,k}(x) B_{v,l}(y) B_{w,m}(z) a_{u,v,w}$$

with $B_{u,k}, B_{v,l}, B_{w,m}$ univariate B-splines. Correspondingly, this spline is of order k in x , of order l in y , and of order m in z .

Next question:

For starting the first draft of paper (Using KAN in SMLM) which other steps should be taken in both (experimental and theoretical)?